

01_staging_empresas_normalizadas

May 9, 2026

1 01. Staging de Empresas Normalizadas

1.1 Goal

Create clean company name tables from both sources and build the unified company universe for downstream matching.

1.2 Inputs

- leads.xlsx (esperado en la raíz del proyecto; fallback: 01_data_ingestion_enrichment/leads.xlsx)
- proyectos_empresa.xlsx (esperado en la raíz del proyecto; fallback: 01_data_ingestion_enrichment/proyectos_empresa.xlsx)

1.3 Outputs (se guardan en 02_data_cleaning/outputs/)

- leads_companies_clean.csv
- horas_empresas_clean.csv
- empresas_universe_compilado.csv

```
[6]: from pathlib import Path
from typing import Iterable, Optional

import pandas as pd

try:
    from IPython.display import display # Jupyter / VS Code Notebooks
except Exception:
    def display(x): # fallback minimal
        print(x)

def find_project_root(start: Optional[Path] = None) -> Path:
    """Encuentra la raíz del proyecto sin depender frágilmente del cwd."""
    start = (start or Path.cwd()).resolve()
    for p in [start, *start.parents]:
        if (p / "01_data_ingestion_enrichment").is_dir() and (p / "02_data_cleaning").is_dir():
            return p
    raise FileNotFoundError(
```

```

        "No se pudo localizar la raíz del proyecto.\n"
        "Sugerencia: abre la carpeta raíz del repo en VS Code y vuelve a
↪ejecutar.\n"
        f"Directorio actual (cwd): {start}"
    )

def pick_existing(candidates: Iterable[Path], *, label: str) -> Path:
    candidates = [Path(p) for p in candidates]
    for p in candidates:
        if p.exists():
            return p
    tried = "\n".join([f" - {p.resolve()}" for p in candidates])
    raise FileNotFoundError(f"No se encontró {label}. Rutas probadas:\n{tried}")

def ensure_dir(dir_path: Path) -> Path:
    dir_path = Path(dir_path)
    dir_path.mkdir(parents=True, exist_ok=True)
    return dir_path

def read_excel_checked(path: Path, **kwargs) -> pd.DataFrame:
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(f"No existe el archivo Excel esperado: {path.
↪resolve()}")
    try:
        return pd.read_excel(path, **kwargs)
    except ImportError as e:
        raise ImportError(
            "No puedo leer .xlsx porque falta un motor (normalmente `openpyxl`).
↪ "
            "Instala con: pip install openpyxl"
        ) from e

def save_df_csv(
    df: pd.DataFrame,
    out_path: Path,
    *,
    index: bool = False,
    encoding: str = "utf-8-sig",
) -> Path:
    out_path = Path(out_path)
    ensure_dir(out_path.parent)
    df.to_csv(out_path, index=index, encoding=encoding)
    print(f"[OK] Guardado: {out_path.resolve()}")
    print(f"      shape: {df.shape}")
    return out_path

```

```

ROOT_DIR = find_project_root()
INGESTION_DIR = ROOT_DIR / "01_data_ingestion_enrichment"
CLEANING_DIR = ROOT_DIR / "02_data_cleaning"
OUTPUT_DIR = ensure_dir(CLEANING_DIR / "outputs")

# Inputs esperados: preferir raíz del proyecto; fallback a 02_data_cleaning/ y
↳ 01_data_ingestion_enrichment/
LEADS_FILE = pick_existing(
    [ROOT_DIR / "leads.xlsx", CLEANING_DIR / "leads.xlsx", INGESTION_DIR /
↳ "leads.xlsx"],
    label="leads.xlsx (input)",
)
HORAS_FILE = pick_existing(
    [ROOT_DIR / "proyectos_empresa.xlsx", CLEANING_DIR / "proyectos_empresa.
↳ xlsx", INGESTION_DIR / "proyectos_empresa.xlsx"],
    label="proyectos_empresa.xlsx (input)",
)

print("ROOT_DIR      ->", ROOT_DIR)
print("INGESTION_DIR ->", INGESTION_DIR)
print("OUTPUT_DIR    ->", OUTPUT_DIR.resolve())
print("LEADS_FILE     ->", LEADS_FILE.resolve())
print("HORAS_FILE     ->", HORAS_FILE.resolve())

```

```

ROOT_DIR      -> E:\TESIS MAESTRIA\Desarrollo_clustering_maestria
INGESTION_DIR -> E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment
OUTPUT_DIR    -> E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\outputs
LEADS_FILE     -> E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\leads.xlsx
HORAS_FILE     -> E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\proyectos_empresa.xlsx

```

```

[7]: # === Carga de datasets (inputs) ===
# - Usa N_FILAS_PRUEBA para iterar rápido (None = dataset completo).
N_FILAS_PRUEBA = None # p.ej. 500 para prueba rápida

df_leads = read_excel_checked(LEADS_FILE, nrows=N_FILAS_PRUEBA)
df_horas = read_excel_checked(HORAS_FILE, nrows=N_FILAS_PRUEBA)

print("[INPUT] Leads:", df_leads.shape, "|", LEADS_FILE.resolve())
print("[INPUT] Horas:", df_horas.shape, "|", HORAS_FILE.resolve())

```

```

[INPUT] Leads: (440, 15) | E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\leads.xlsx
[INPUT] Horas: (412, 18) | E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\proyectos_empresa.xlsx

```

```
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-  
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no  
default style, apply openpyxl's default  
warn("Workbook contains no default style, apply openpyxl's default")
```

1.4 Nota de diseño

- Este notebook es **standalone**: lee `leads.xlsx` y `proyectos_empresa.xlsx` desde la raíz del proyecto (con fallback) y escribe únicamente a `02_data_cleaning/outputs/`.
- No depende de dataframes creados por otros notebooks; el intercambio con pasos posteriores es solo vía archivos en `outputs/`.
- Siguiente notebook que consume estos outputs: `02_matching_exacto_scvs.ipynb` (usa `leads_companies_clean.csv` y `horas_empresas_clean.csv`).

1.5 9. Universo unificado de empresas (LEADS HORAS) y verificación contra SCVS

Objetivo: como no hay coincidencias exactas entre ambas fuentes, construimos el **universo unificado** de empresas (a partir de ambas) y evaluamos cobertura en una fuente externa (SCVS / Superintendencia de Compañías).

- Esta sección exporta los CSV de staging a `02_data_cleaning/outputs/`:
 - `leads_companies_clean.csv`
 - `horas_empresas_clean.csv`
 - `empresas_universe_compilado.csv`

```
[8]: # === 9.a Limpieza LEADS (Company) ===  
# Resultado esperado: DataFrame `leads_companies_clean` con Company raw +  
    ↪ normalizada (distinct por normalizada)  
  
import re  
import unicodedata  
import numpy as np  
import pandas as pd  
  
def _strip_accents(text: str) -> str:  
    return "".join(  
        ch for ch in unicodedata.normalize("NFKD", text)  
        if not unicodedata.combining(ch)  
    )  
  
def normalize_company_name(value) -> str:  
    """Normaliza nombres de empresa para comparación (no para mostrar)."""  
    if value is None:  
        return ""  
    if isinstance(value, float) and np.isnan(value):  
        return ""  
  
    s = str(value).strip()
```

```

if not s:
    return ""

s = _strip_accents(s)
s = s.upper()
s = re.sub(r"[^A-Z0-9 ]+", " ", s)

# Quitar sufijos/palabras frecuentes que distorsionan el matching
s = re.sub(
    r"\b(SA|S*s*A|S\A\.|S\A\S\.|SAS|LTDA|CIA|C\.?IA\.|
↳COMPANIA|COMPAÑIA|CORP|INC|LLC|C\.L\.|C\.?LTDA\.|&|Y)\b",
    " ",
    s,
)
s = re.sub(r"\b(DE|DEL|LA|EL|LOS|LAS)\b", " ", s)
s = re.sub(r"\s+", " ", s).strip()
return s

if "df_leads" not in globals():
    raise NameError("No existe `df_leads` en memoria. Ejecuta primero la celda_
↳de 'Carga de datasets'.")

if "Company" not in df_leads.columns:
    raise KeyError("La columna `Company` no existe en `df_leads`.")

_company_raw = (
    df_leads["Company"]
    .astype("string")
    .fillna("")
    .map(lambda x: x.strip())
    .replace("", pd.NA)
    .dropna()
)

leads_companies_clean = (
    pd.DataFrame({"Company_raw": _company_raw})
    .assign(Company_norm=lambda d: d["Company_raw"].map(normalize_company_name))
    .query("Company_norm != ''")
    .drop_duplicates(subset=["Company_norm"], keep="first")
    .sort_values("Company_norm")
    .reset_index(drop=True)
)

print("[LEADS] Filas originales (no vacías):", int(_company_raw.shape[0]))
print("[LEADS] Empresas distinct (por Company_norm):",
↳int(leads_companies_clean.shape[0]))

```

```
display(leads_companies_clean.head(20))

# --- Export CSV ---
_ = save_df_csv(leads_companies_clean, OUTPUT_DIR / "leads_companies_clean.csv")
```

[LEADS] Filas originales (no vacías): 440

[LEADS] Empresas distinct (por Company_norm): 326

	Company_raw	Company_norm
0	7-ELEVEN MEXICO	7 ELEVEN MEXICO
1	ABBOTT	ABBOTT
2	Abbvie	ABBVIE
3	ACCO BRANDS	ACCO BRANDS
4	ACH FOOD COMPANIES, INC	ACH FOOD COMPANIES
5	ACTINVER	ACTINVER
6	ADAMANTINE	ADAMANTINE
7	AFP Genesis	AFP GENESIS
8	AIG	AIG
9	Akros	AKROS
10	ALCIONE MX	ALCIONE MX
11	AlimentosAlConsumidor	ALIMENTOSALCONSUMIDOR
12	ALLIANZ MÉXICO	ALLIANZ MEXICO
13	ALPEZZI CHOCOLATE	ALPEZZI CHOCOLATE
14	Alpina	ALPINA
15	Alvarez Barba S.A	ALVAREZ BARBA
16	AMBROSIA	AMBROSIA
17	ANHEUSER -BUSCH INBEV	ANHEUSER BUSCH INBEV
18	La Anita	ANITA
19	APOTEX	APOTEX

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning
 \outputs\leads_companies_clean.csv
 shape: (326, 2)

```
[9]: # === 9.b Limpieza HORAS (EMPRESA) ===
# Resultado esperado: DataFrame `horas_empresas_clean` con EMPRESA raw +
#   ↪normalizada (distinct por normalizada)

import pandas as pd

if "df_horas" not in globals():
    raise NameError("No existe `df_horas` en memoria. Ejecuta primero la celda
    ↪de 'Carga de datasets'.")

if "EMPRESA" not in df_horas.columns:
    raise KeyError("La columna `EMPRESA` no existe en `df_horas`.")

if "normalize_company_name" not in globals():
```

```

        raise NameError("No existe `normalize_company_name` en memoria. Ejecuta_
↪primero la celda 9.a.")

_empresa_raw = (
    df_horas["EMPRESA"]
    .astype("string")
    .fillna("")
    .map(lambda x: x.strip())
    .replace("", pd.NA)
    .dropna()
)

horas_empresas_clean = (
    pd.DataFrame({"EMPRESA_raw": _empresa_raw})
    .assign(EMPRESA_norm=lambda d: d["EMPRESA_raw"].map(normalize_company_name))
    .query("EMPRESA_norm != ''")
    .drop_duplicates(subset=["EMPRESA_norm"], keep="first")
    .sort_values("EMPRESA_norm")
    .reset_index(drop=True)
)

print("[HORAS] Filas originales (no vacías):", int(_empresa_raw.shape[0]))
print("[HORAS] Empresas distinct (por EMPRESA_norm):", int(horas_empresas_clean.
↪shape[0]))

display(horas_empresas_clean.head(20))

# --- Export CSV ---
save_df_csv(horas_empresas_clean, OUTPUT_DIR / "horas_empresas_clean.csv")

# --- CSV compilado de empresas (LEADS HORAS) ---
if "leads_companies_clean" not in globals():
    raise NameError("No existe `leads_companies_clean` en memoria. Ejecuta_
↪primero la sección 9.a (Limpieza LEADS).")

leads_u = (
    leads_companies_clean
    .rename(columns={"Company_raw": "name_raw", "Company_norm": "name_norm"})
    .assign(source="LEADS")
    [["source", "name_raw", "name_norm"]]
)

horas_u = (
    horas_empresas_clean
    .rename(columns={"EMPRESA_raw": "name_raw", "EMPRESA_norm": "name_norm"})
    .assign(source="HORAS")
    [["source", "name_raw", "name_norm"]]
)

```

```

universe_all = pd.concat([leads_u, horas_u], ignore_index=True)
universe_distinct = (
    universe_all
    .groupby("name_norm", as_index=False)
    .agg(
        name_raw=("name_raw", "first"),
        sources=("source", lambda s: ",".join(sorted(set(map(str, s))))),
        n_rows=("source", "size"),
    )
    .sort_values("name_norm")
    .reset_index(drop=True)
)

save_df_csv(universe_distinct, OUTPUT_DIR / "empresas_universe_compilado.csv")
display(universe_distinct.head(20))

```

[HORAS] Filas originales (no vacías): 412

[HORAS] Empresas distinct (por EMPRESA_norm): 119

	EMPRESA_raw	EMPRESA_norm
0	3dpharma	3DPHARMA
1	Adium	ADIUM
2	Almexa	ALMEXA
3	Alper Seguros	ALPER SEGUROS
4	Arauco	ARAUCO
5	Aseguradora del Sur	ASEGURADORA SUR
6	Asesoría y Control	ASESORIA CONTROL
7	AutoShare	AUTOSHARE
8	AVIS	AVIS
9	BAC	BAC
10	Banco del Austro	BANCO AUSTRO
11	Banco Financiero	BANCO FINANCIERO
12	BANRED	BANRED
13	Bekaert	BEKAERT
14	Bemis	BEMIS
15	Bitso	BITSO
16	Bloomin Brands	BLOOMIN BRANDS
17	BMI	BMI
18	Boston BSCI	BOSTON BSCI
19	CEPSA	CEPSA

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\outputs\horas_empresas_clean.csv

shape: (119, 2)

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\outputs\empresas_universe_compilado.csv

shape: (445, 4)

	name_norm	name_raw	sources	n_rows
0	3DPHARMA	3dpharma	HORAS	1
1	7 ELEVEN MEXICO	7-ELEVEN MEXICO	LEADS	1
2	ABBOTT	ABBOTT	LEADS	1
3	ABBVIE	Abbvie	LEADS	1
4	ACCO BRANDS	ACCO BRANDS	LEADS	1
5	ACH FOOD COMPANIES	ACH FOOD COMPANIES, INC	LEADS	1
6	ACTINVER	ACTINVER	LEADS	1
7	ADAMANTINE	ADAMANTINE	LEADS	1
8	ADIUM	Adium	HORAS	1
9	AFP GENESIS	AFP Genesis	LEADS	1
10	AIG	AIG	LEADS	1
11	AKROS	Akros	LEADS	1
12	ALCIONE MX	ALCIONE MX	LEADS	1
13	ALIMENTOSALCONSUMIDOR	AlimentosAlConsumidor	LEADS	1
14	ALLIANZ MEXICO	ALLIANZ MÉXICO	LEADS	1
15	ALMEXA	Almexa	HORAS	1
16	ALPER SEGUROS	Alper Seguros	HORAS	1
17	ALPEZZI CHOCOLATE	ALPEZZI CHOCOLATE	LEADS	1
18	ALPINA	Alpina	LEADS	1
19	ALVAREZ BARBA	Alvarez Barba S.A	LEADS	1

```
[10]: # --- Verificación de outputs generados ---
expected_outputs = [
    OUTPUT_DIR / "leads_companies_clean.csv",
    OUTPUT_DIR / "horas_empresas_clean.csv",
    OUTPUT_DIR / "empresas_universe_compilado.csv",
]

print("\n[CHECK] Outputs esperados en:", OUTPUT_DIR.resolve())
missing = []
for p in expected_outputs:
    if p.exists():
        print(" - OK      ", p.resolve())
    else:
        print(" - MISSING", p.resolve())
        missing.append(p)

if missing:
    raise FileNotFoundError(
        "Faltan outputs esperados. Revisa si ejecutaste todas las secciones del_
↪notebook.\n"
        + "\n".join([str(p.resolve()) for p in missing])
    )
```

```
[CHECK] Outputs esperados en: E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\outputs
```

- OK E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\out
puts\leads_companies_clean.csv
- OK E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\out
puts\horas_empresas_clean.csv
- OK E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\out
puts\empresas_universe_compilado.csv